Sprint 1 Document

Team 52 — Clue! (G6046 Software Engineering)

1. General Information

Dates

Start Date	Sprint End Date
05/03/2026	22/03/2026

People Involved

Name	Role	Status
Thanh Shaw	Board and movement logic, repository setup, early implementation support, process documentation support	Active
Insik Ham	No identifiable submitted contribution beyond limited meeting attendance	Non-contributing
Omar Abu Isbeih	No identifiable submitted contribution beyond limited meeting attendance	Non-contributing
Murat Maimaiti	TBC	Non-responding
Shengyuan Liu	TBC	Non-responding

2. Sprint Goal

The main goal of Sprint 1 was to establish the project foundations. This included choosing the technology stack, setting up the repository and working practices, creating the initial model structure, and beginning implementation of the key game classes needed for Clue!.

Sprint 1 was intended to deliver a compilable starting codebase rather than a complete game. The focus was on the core data model, board representation, cards, dice, murder envelope, player structure and early testing so that later sprints could build on a stable base.

3. Tasks for this Sprint

Task	Assignee	Status
Set up GitHub repository and shared project structure	Thanh	Complete
Agree programming language, tools and initial architecture	Team 52	Complete
Create initial card hierarchy and deck-related classes	Team 52	Complete

Create murder envelope and early accusation-checking support	Team 52	Complete
Create board, room and square model structure	Thanh	Complete
Create player hierarchy and basic game-state structure	Team 52	Complete
Implement fair dice behaviour	Thanh	Complete
Write initial JUnit unit tests for core classes	Thanh	Complete
Document initial design decisions and class responsibilities	Team 52	Partial
Complete Sprint 1 documentation and process record	Team 52	Partial

4. User Stories

- As a player, I want the game to use a standard deck of 21 cards so that every game starts with the correct set of suspects, weapons and rooms.
- As a player, I want one card of each type sealed secretly at the start so that the murder solution is hidden from all players.
- As a player, I want to be dealt cards from the remaining deck so that I can use them to eliminate possibilities during the game.
- As a player, I want the board to represent the rooms and corridors of Archers Avenue so that the game reflects the original layout.
- As a player, I want dice to produce fair rolls so that movement is random and unpredictable.
- As a player, I want to play against an AI opponent so that the game is playable without a full group of human players.

5. Requirements to Fulfil in this Sprint

Functional Requirements

- FR1: The game shall use a standard deck of 21 cards: 6 suspects, 6 weapons and 9 rooms.
- FR2: The deck shall be shuffled randomly and one card of each type placed secretly in the murder envelope before dealing the remainder to players.
- FR3: The game shall support between 2 and 6 players including AI players.
- FR4: The board shall represent the ground floor of Archers Avenue with 9 rooms, corridors, starting squares and blocked squares.
- FR5: The game shall use two fair six-sided dice with an equal probability of landing on any face.
- FR6: Players shall be placed on designated starting squares at the beginning of the game.

Non-Functional Requirements

- NFR1: The codebase shall follow Java OO principles including inheritance, polymorphism, high cohesion and low coupling.
- NFR2: All public classes and methods shall include JavaDoc comments.
- NFR3: The project shall use Java 21 and be version-controlled on GitHub with commits from active team members.

6. Design Notes

Architecture

Sprint 1 focused on establishing the class hierarchy and package structure. The project was organised into four packages: model (board, rooms, squares, dice, game state), cards (card hierarchy, deck, murder envelope), players (abstract player, human and AI subclasses) and gui (deferred to Sprint 3).

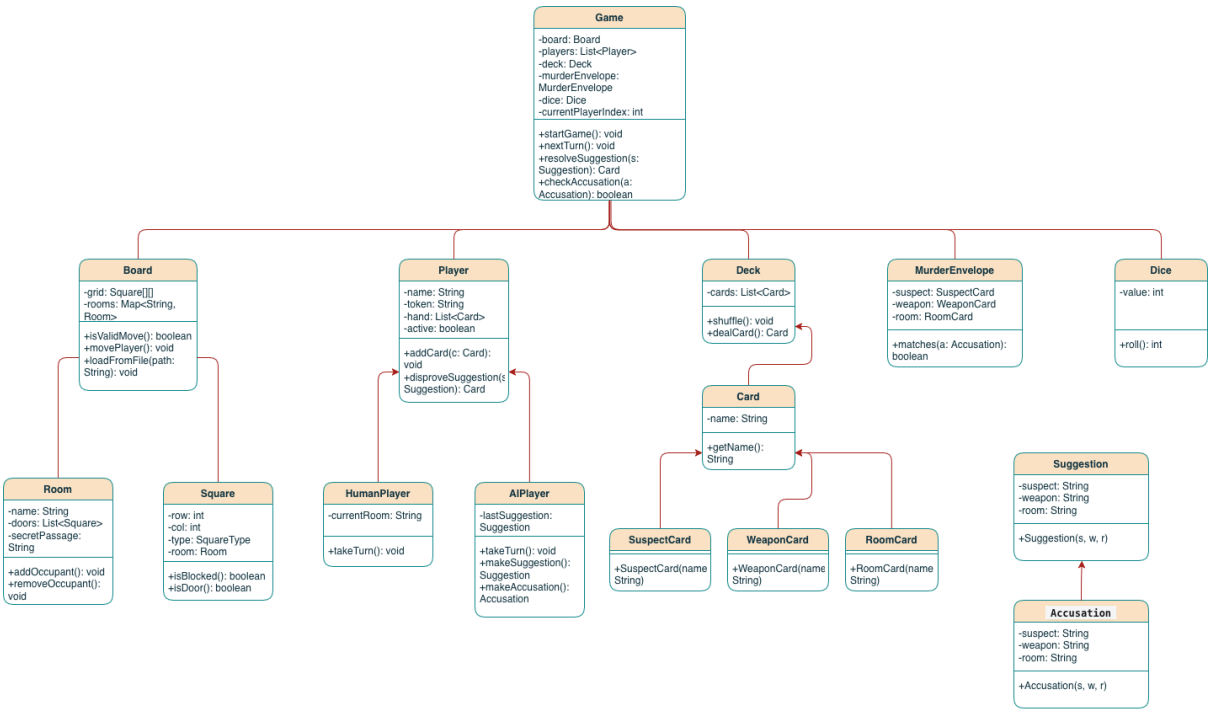
Class	Responsibility
Card / SuspectCard / WeaponCard / RoomCard	Abstract card base class with three concrete subclasses for OO hierarchy
Deck	Creates all 21 cards, supports shuffling and dealing
MurderEnvelope	Holds and verifies the three hidden solution cards
Board	24x25 grid of Square objects representing the game board
Room	Room name, door squares, secret passage references and occupants
Square	Single grid cell with type (CORRIDOR, ROOM, BLOCKED, START) and occupancy
Dice	Fair two-die roll using java.util.Random
Player / HumanPlayer / AIPlayer	Abstract player with hand, position and turn behaviour: subclasses for human and AI
Game	Top-level controller: setup, dealing, turn order and win conditions

Key Design Decisions

- Java 21 with JavaFX selected as the technology stack: supports OO design, JavaDoc and cross-platform deployment.
- Abstract Card class with three concrete subclasses: demonstrates OO inheritance and polymorphism directly relevant to the marking criteria.
- Abstract Player class with HumanPlayer and AIPlayer subclasses: allows polymorphic turn handling and clean separation of GUI-driven and autonomous logic.
- Board represented as a 2D array of Square objects: simple to index and supports movement validation in later sprints.
- Board layout hard-coded to match the classic Clue! board image from the coursework spec with 9 rooms in an initial layout.
- Secret passages implemented between Study and Kitchen and between Conservatory and Lounge as per the spec.

Diagrams

Class diagram (Sprint 1):



7. Testing Completed in this Sprint

Sprint 1 introduced unit testing using JUnit 5. Tests were written alongside implementation to verify the core model classes before Sprint 2 built on top of them.

Test Area	Coverage
Dice	roll() always returns 2-12; rollSingle() always returns 1-6 over 1000 rolls
Deck	Exactly 21 cards created; correct count of 6 suspects, 6 weapons and 9 rooms
MurderEnvelope	verify() returns true on correct match; false on wrong suspect, weapon or room; isSealed() correct before and after seal()
Game setup	Envelope sealed after startGame(); each player dealt correct number of cards; no overlap between hand cards and envelope cards

By the end of Sprint 1 there were 11 passing JUnit tests covering the core model classes. All tests passed with Java 21 and remained green throughout subsequent sprints.

8. Sprint Changelog

- Created private GitHub repository with shared folder structure (src, tests, docs).
- Agreed technology stack: Java 21, IntelliJ IDEA, JUnit 5, GitHub, JavaDoc.
- Implemented Card abstract class and SuspectCard, WeaponCard, RoomCard subclasses.
- Implemented Deck class: 21 cards, shuffle and round-robin deal.

- Implemented MurderEnvelope: seal() and verify() methods.
- Implemented Board class: 24x25 grid, 9 rooms in an initial layout, secret passages, doors and blocked squares.
- Implemented Room and Square classes with type enum and occupancy tracking.
- Implemented Dice class: fair two-die roll.
- Implemented Game class: startGame() dealing and token placement, nextTurn() skeleton.
- Implemented Player abstract class and HumanPlayer and AIPlayer subclasses.
- Implemented Suggestion and Accusation value objects.
- Wrote 11 JUnit 5 unit tests: all passing.

9. Post-Sprint Review

Did the sprint achieve its goal?

Yes. Sprint 1 delivered a compilable Java codebase with the main core model classes in place and an initial set of passing unit tests. The foundations were stable enough to support Sprint 2 game logic work without requiring significant rework of the class structure.

What went well?

- The repository and project structure were set up cleanly and consistently from the start.
- The OO class hierarchy was well-designed and remained stable throughout the project.
- Early JUnit tests gave confidence that core classes were working before more complex logic was added.
- JavaDoc was added to all classes from the beginning, reducing documentation debt later.

What didn't go well?

- Team contribution was uneven from the outset: two members were non-responding and two others attended only minimally and made no contribution.
- One team member was ill during the sprint, which delayed some implementation work and caused the sprint end date to be extended from 19 March to 22 March.
- Some design documentation and sprint records were not produced during the sprint and required retrospective completion.

What would we do differently?

- Establish clearer contribution expectations and communication channels earlier in the project.
- Keep sprint documentation updated in parallel with implementation rather than retrospectively.
- Raise team engagement issues with the module team earlier rather than waiting.

Carry-over to Sprint 2

- Implement the full game loop, turn order and movement logic.
- Build the board layout properly with room boundaries, doors and movement validation.
- Implement suggestion and accusation game flow.
- Deliver a working console prototype of the game.
- Expand test coverage to cover movement and game flow.